Basic MQL and Document Modeling

MongoDB stores documents.
Queries find them. Writes change them.

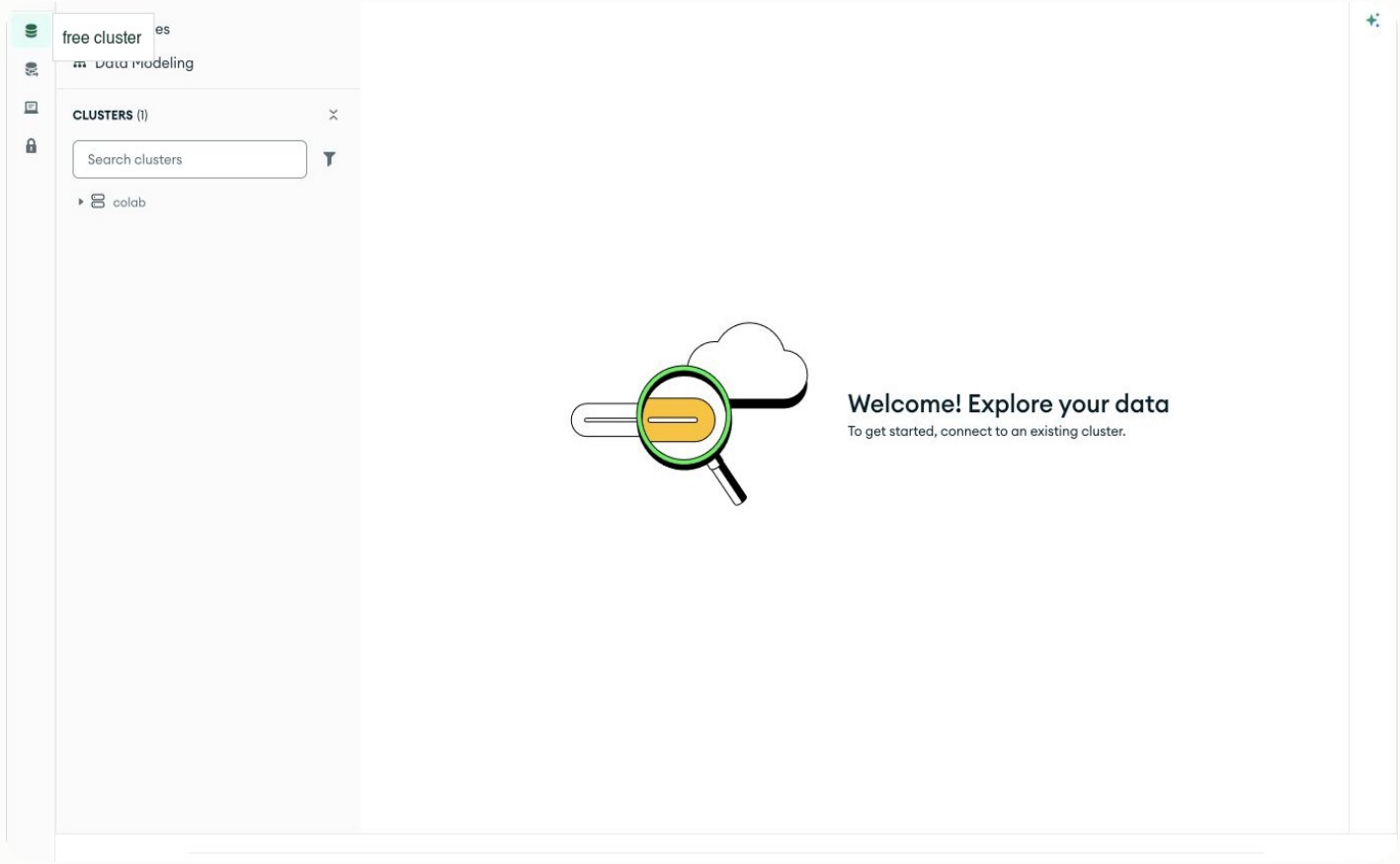
Day 1: reading and changing documents
Day 2: creating documents and choosing what belongs together

SQL concepts and Python MongoDB operations

Purpose	SQL idea	Python / PyMongo
Choose a source	FROM tickets	database["tickets"]
Choose records	WHERE status = 'open'	{"status": "open"}
Choose output fields	SELECT ticket_id	{"ticket_id": 1, "_id": 0}
Order and restrict	ORDER BY ticket_id LIMIT 2	.sort("ticket_id", 1).limit(2)

A filter selects documents. A projection selects the fields in each result.
The examples use tickets as the name of a collection.

MongoDB, Atlas, and the client program



MongoDB: database software

Atlas: managed MongoDB service

Database: named collections

Collection: stored documents

An Atlas administration screen. The client program sends queries to the database service.

Atlas interface captured August 25, 2026. Names redacted.

The program connects to the database

Connection requirement	The concept
Server address	Where the database service can be reached
Network access	Whether the client computer can reach that service
Database identity	Which database user the program authenticates as
Permissions	Which collections and operations that user may access

A notebook runs code in a runtime, which may be on another computer.
That runtime sends the database request. The browser displays the result.

Browser activities and notebook resources

JSON, Python, and shell notation

Value or operation	Python / PyMongo	mongosh
Null value	None	null
Boolean value	True / False	true / false
Find one document	tickets.find_one({...})	db.tickets.findOne({...})
Update-result property	result.modified_count	result.modifiedCount
Typed date	datetime(..., tzinfo=timezone.utc)	ISODate("2026-02-01T23:10:00Z")

Day 1 examples use Python with PyMongo. Day 2 examples use mongosh.
Both send operations to MongoDB. A quoted date is text until converted to a date type.

Status and priority describe different facts

Ticket	Category	Priority	Status
1001	streetlight	high	open
1002	sanitation	medium	in_progress
1003	water	urgent	resolved
1004	parks	low	new
1005	sanitation	high	resolved
1006	streetlight	medium	in_progress

Illustrative tickets. Status records workflow. Priority records importance.
A resolved ticket is outside a queue of work that still needs attention.

A document stores related facts

```
{
  "_id": "account-7",
  "owner": "Ari",
  "status": "open",
  "balance": 25.50
}
```

One document describes one account.
A field pairs a name with a value of a particular type.

Here _id is a supplied string identifier. MongoDB also generates ObjectIds.
A collection holds multiple documents with meanings defined by the application.

Nested objects and arrays

```
{
  "owner": "Ari",
  "address": {"city": "New York", "country": "US"},
  "tags": ["student", "savings"],
  "events": [
    {"type": "opened", "actor_role": "resident"}
  ]
}
```

An object groups named fields. An array holds a list.
Array items can be simple values or objects of their own.

address.city names a nested field. tags contains two strings.
Each item in events has its own type and actor_role fields.

An absent field and a null value

```
// Three documents, shown in JSON-style notation
{ "ticket_id": 1, "assignee": null }
{ "ticket_id": 2 }
{ "ticket_id": 3, "assignee": "Ari" }
```

Ticket 1 contains an assignee field whose value is null.
Ticket 2 has no assignee field. Ticket 3 names an assignee.

MongoDB’s {assignee: null} filter matches tickets 1 and 2.
An existence condition distinguishes a missing field from a present one.

A filter and a projection

```
# Collection data relevant to this question:
# {"ticket_id": 1001, "status": "open", "priority": "high"}
# {"ticket_id": 1003, "status": "resolved", "priority": "urgent"}
filter_doc = {"status": "open", "priority": {"$in": ["high", "urgent"]}}
projection = {"_id": 0, "ticket_id": 1, "status": 1}
list(tickets.find(filter_doc, projection))
```

```
Result: [{"ticket_id": 1001, "status": "open"}]
```

Both filter conditions must hold. The resolved ticket fails status = open.
The projection leaves priority out of the result while the stored value remains.

Sorting and limiting a result

```
# Source: ticket_id, status, opened_at (stored dates, abbreviated)
# 1001, open,        2026-02-01
# 1002, in_progress, 2026-02-02
# 1003, resolved,    2026-02-03
# 1006, in_progress, 2026-02-04
cursor = tickets.find({"status":{"$in":["open","in_progress"]}},
                      {"_id":0,"ticket_id":1})
cursor.sort([("opened_at",-1),("ticket_id",1)]).limit(2)
```

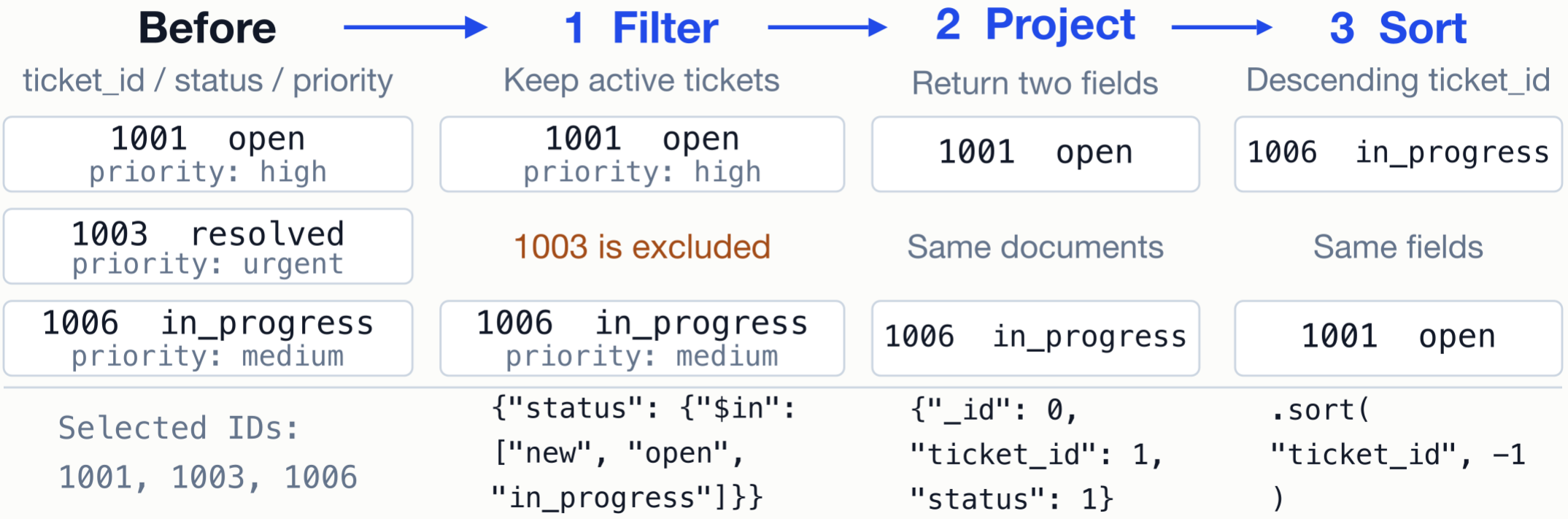
Result: 1006, then 1002. Ticket 1001 falls beyond the limit.
Ticket 1003 fails the status filter. The newest date comes first.

A cursor supplies the results as the program reads them. limit(2) changes no stored records.

Filtering, projection, and ordering

Filter chooses documents. Projection chooses fields. Sort orders results.

Three selected tickets from the six-ticket fixture; other fields are omitted here.



These are conceptual steps. The stored documents are unchanged.

Continue to nested fields: slide 13

Nested fields and simple arrays

```
# Source documents (all fields needed for these queries):
# {"ticket_id":1001,"requester":{"name":"Ari"},"tags":["safety"]}
# {"ticket_id":1005,"requester":{"name":"Bo"},"tags":["safety"]}
projection = {"_id": 0, "ticket_id": 1}
list(tickets.find({"requester.name": "Ari"}, projection))
list(tickets.find({"tags": "safety"}, projection))
```

Nested-field query: 1001. Array-value query: 1001 and 1005.
The dotted path reaches a name. The tag condition finds a value in an array.

Two conditions can match different events

Ticket 1001 event	type	actor_role
5001	created	resident
5002	assigned	agent

```
wrong_question = {
    "events.type": "created",
    "events.actor_role": "agent"
}
```

The shown ticket matches: created is in event 5001, and agent is in event 5002.

Static diagram on slide 16

$elemMatch requires one qualifying event

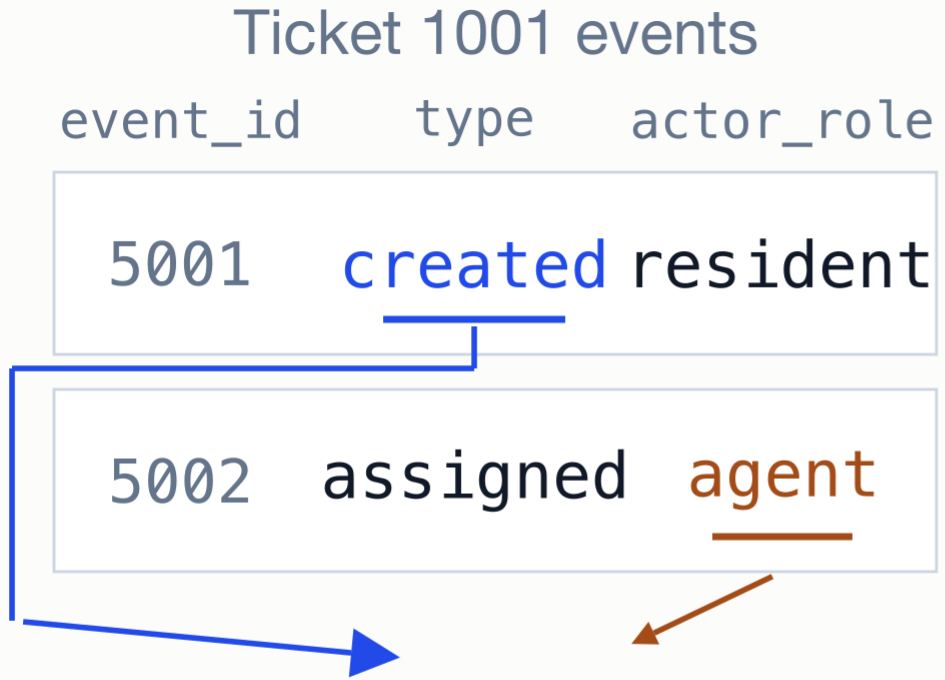
```
# One ticket: 1001. Its complete events array is:
# [{"type":"created", "actor_role":"resident"},
#  {"type":"assigned", "actor_role":"agent"}]
query = {"events": {"$elemMatch": {
  "type": "created", "actor_role": "agent"
}}}
list(tickets.find(query, {"_id": 0, "ticket_id": 1}))
```

Result: [] because neither event contains both requested values.

For type = assigned and actor_role = agent, this ticket qualifies.
$elemMatch in the filter selects documents, rather than removing other events.

Static diagram on slide 16

Array conditions and one matching event



Separate dotted conditions

Both values occur somewhere
Ticket matches



`$elemMatch`

No event contains both values
Ticket does not match

Continue to slide 17

The parts of a document update

```
# Before: one document with these fields
# {"ticket_id": 1099, "status": "new", "events": []}
tickets.update_one(
  {"ticket_id": 1099},
  {"$set": {"status": "in_progress"}}
)
```

The filter identifies the document. $set names the field and new value. Afterward: ticket_id 1099, status in_progress, events still empty.

Updating selected fields preserves the other fields. Replacing a document is a different operation.

A repeated $set can match without changing

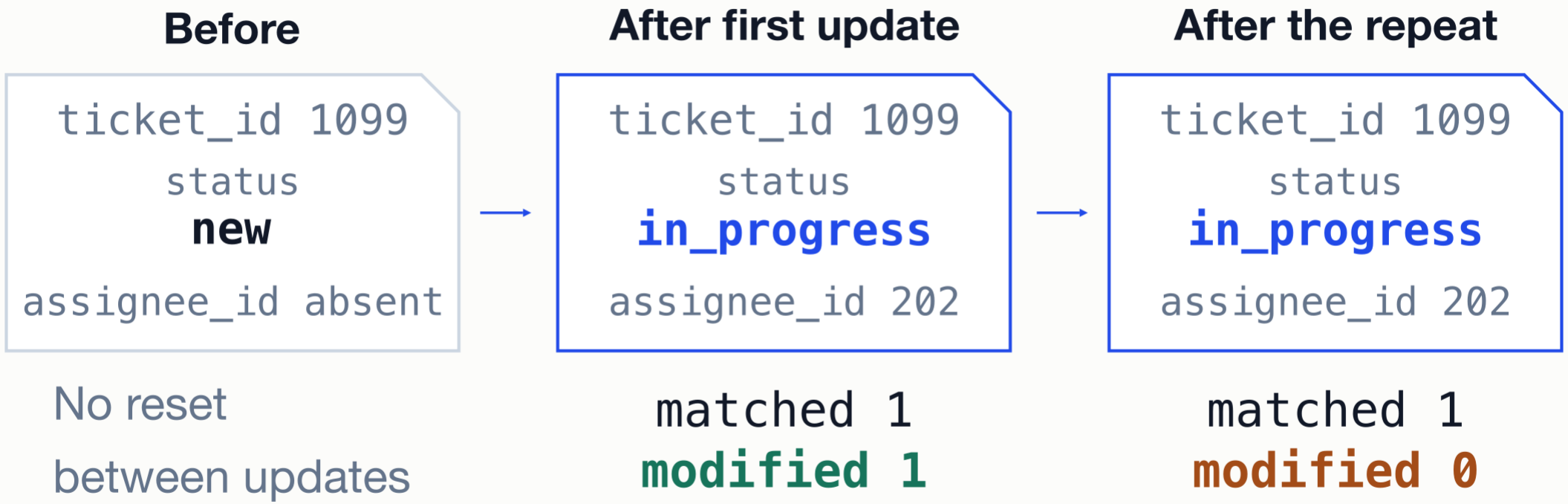
```
# Before: {"ticket_id":1099, "status":"new"}; no assignee_id field
result = tickets.update_one(
  {"ticket_id": 1099},
  {"$set": {"status": "in_progress", "assignee_id": 202}}
)
print(result.matched_count, result.modified_count)
```

Execution	Matched	Modified	Stored status
First	1	1	in_progress
Same update again	1	0	in_progress

Static diagram on slide 19

The first update and the repeat

```
$set  status = "in_progress", assignee_id = 202
```



The second filter still matches. The requested values are already stored.

Continue to slide 20

An expected-state condition can prevent a write

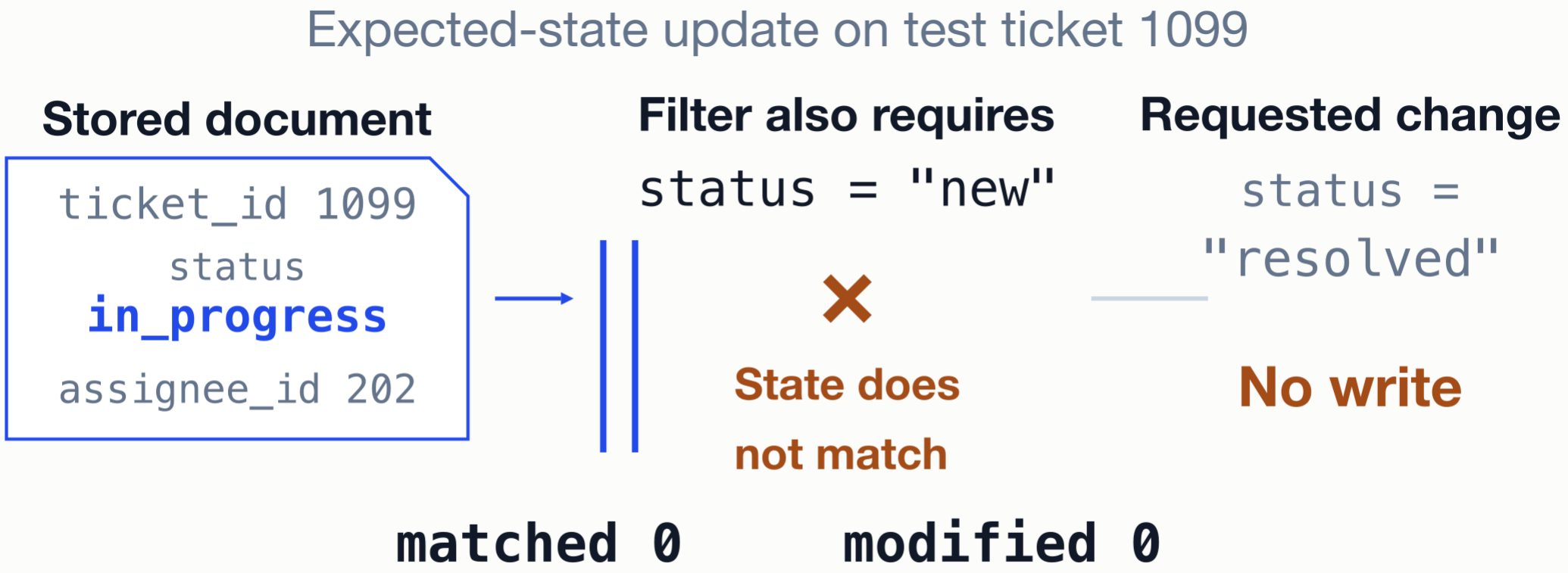
```
# Current document: {"ticket_id":1099, "status":"in_progress"}
result = tickets.update_one(
  {"ticket_id": 1099, "status": "new"},
  {"$set": {"status": "resolved"}}
)
print(result.matched_count, result.modified_count)
```

Result: 0 matched, 0 modified. Status remains in_progress.
The ticket exists, but its current status fails the filter.

The application can reload the current record before deciding what to do next.

Static diagram on slide 21

The expected state in an update filter



Status stays in_progress. Reload the document before deciding what to do next.

Continue to slide 22

Appending an event and repeating the request

```
# Before: {"ticket_id":1099, "events":[]}
result = tickets.update_one(
  {"ticket_id": 1099, "events.event_id": {"$ne": 5999}},
  {"$push": {"events": {
    "event_id": 5999, "type": "status_changed", "actor_role": "agent"
  }}}
)
print(result.matched_count, result.modified_count)
```

First request: 1 matched, 1 modified. Repeat: 0 matched, 0 modified.
After either request, events contains one copy of event 5999.

$push appends an array item. The filter requires that this event ID be absent.
An ID field inside an array has no automatic uniqueness rule.

Deletion with an exact target

```
# Source collection contains these two tickets:
# {"ticket_id":1099,"status":"in_progress"},
{"ticket_id":1001,"status":"open"}
target = {"ticket_id": 1099}
result = tickets.delete_one(target)
print(result.deleted_count)
print(tickets.find_one(target))
```

Result: 1 deleted, then None on read-back. Ticket 1001 remains.
Repeating the same deletion reports 0.

delete_one removes one matching document. delete_many removes every match.
An empty filter matches every document, so the filter defines the scope of a deletion.

Lab 1 applies the lecture concepts

The handout supplies the dataset and setup instructions.

Build a queue of active lighting and sanitation tickets.
Compare array filters. Observe a repeated update.
Use the returned counts and documents to explain what happened.

Submit your completed notebook and its short results record.
Follow the handout for setup, recovery steps, and cleanup.

Creating and reading a document

A new account becomes a stored document.
A read-back shows what MongoDB saved.

CRUD means create, read, update, and delete.
The following examples use the MongoDB shell, mongosh.
The data, commands, and results appear together.

Creating one document with insertOne

Our accounts collection starts empty. Each account will be one document.

MongoDB shell command

```
const saved = db.accounts.insertOne({
  owner: "Ari", status: "open"
})
saved
```

What the shell returns

```
{
  acknowledged: true,
  insertedId: ObjectId("6aed03ee82c481486b6c4e15")
}
```

insertedId identifies this new document. acknowledged reports the write acknowledgment.

Read the account and insert a batch

Use the ID from the write to look up Ari’s account in the same collection.

Read the stored document in mongosh

```
db.accounts.findOne({ _id: saved.insertedId })
// Result
{ _id: ObjectId("6aed03ee82c481486b6c4e15"),
  owner: "Ari", status: "open" }
```

Add two accounts in one call

```
db.accounts.insertMany([
  { owner: "Bo", status: "open" },
  { owner: "Cam", status: "open" }
])
```

One Ari document plus two new documents gives three accounts after one successful call.

Status and history in one document update

```
// Before: {ticket_id:1001, status:"open", events:[]}
db.tickets.updateOne(
  {ticket_id: 1001, status: "open"},
  {$set: {status: "in_progress"},
   $push: {events: {event_id: 5099, type: "status_changed"}}}
)
```

After: status is in_progress and events contains event 5099.
Both changes form one atomic document write. Repeat: 0 matched, 0 modified.

Atomic means the write applies all its changes together. This guarantee covers one document.

Current contact details and historical snapshots

Stored fact	Owner	Effect of a contact correction
requester_id: 101	Ticket stores the relationship	ID stays the same
Current email for user 101	Authoritative user document	Update one user record
requester_name_at_open	Ticket owns a historical snapshot	Keep it only if the historical meaning is intended

The page can read the user by ID for current contact details.
A copied current email would introduce another value to maintain.

A reference on each event allows independent growth

```
{"ticket_id": 1001, "requester_id": 101, "status": "open"}
```

```
{
  "event_id": 5002,
  "ticket_id": 1001,
  "event_type": "assigned",
  "event_at": "2026-02-02T14:05:00Z"
}
```

Each event identifies its ticket. Adding an event leaves the ticket’s size unchanged.
The displayed timestamp is JSON text. Stored date values support chronological queries.

Two displayed events, three retained events

Illustrative records for one page. All event times below share the same date.

Source records

```
ticket: {ticket_id:1001, requester_id:101, status:"open"}
person: {user_id:101, display_name:"Ari"}
history for ticket 1001: 5001 at 09:00, 5002 at 10:00, 5003 at 11:00
// event_at stores a BSON date, event_id identifies each event
```

The history query in mongosh

```
db.history.find(
  {ticket_id: 1001}, {_id: 0, event_id: 1}
).sort({event_at: -1, event_id: -1}).limit(2)
```

The page shows open, Ari, and events 5003 then 5002. Event 5001 remains stored.

Growing history and a bounded preview

Representation	What grows with each new event	Page-read consequence
All events embedded	The ticket document	One read, increasing document size
All events separate	The number of event documents	Ticket plus bounded history query
Separate history and a recent copy	History, but recent copy stays capped	Convenient preview with refresh work

A BSON document has a 16 MiB size limit.
A design can become awkward well before that limit.

A capped copy needs an explicit refresh and stale-data policy.

Choosing document boundaries

A shared user document gives contact corrections one home. Separate event documents let the history grow independently.

The page retrieves related records. If it requires one consistent view across them, the application must coordinate those reads.

Workloads matter in PostgreSQL too. PostgreSQL supports arrays and JSON. Neither a SQL table nor a MongoDB collection chooses a good model for us.

Week 10 Lab 2

Launch Lab 1: Perform CRUD Operations and Design Effective Queries.

Open the student handout for required Part A.
Submit one Brightspace entry with your workspace, insert and read-back, and changed filter.

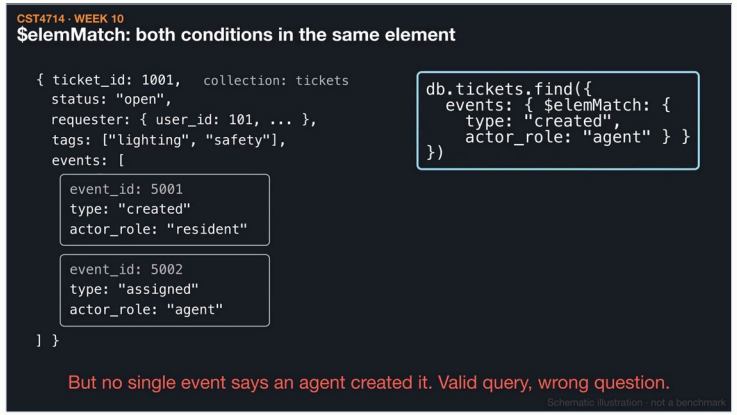
Optional Part B. Chapter 10 reading response
If you write it, add it to the same Brightspace entry.

The required lab record

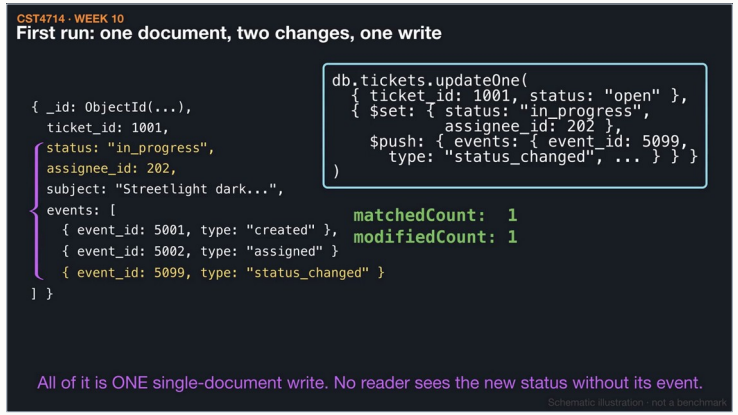
Record the workspace you reached, what you inserted and read back, and how changing a filter affected the result. A clear record connects each command to the evidence it produced.

Submit required Part A as one Brightspace text entry.
The Chapter 10 reading response in Part B is optional.

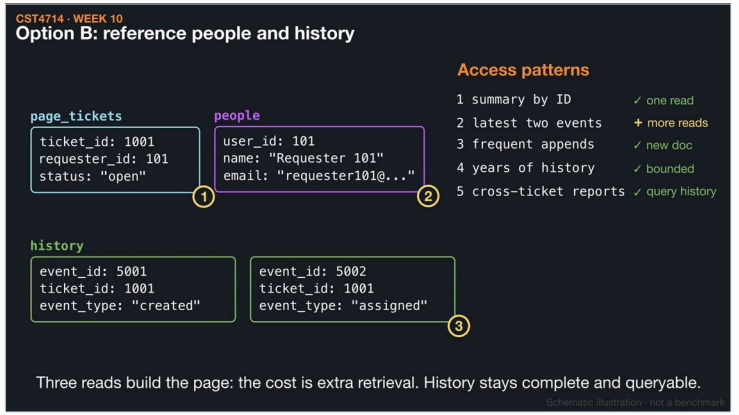
This week's videos



▶ Querying inside documents: arrays and $elemMatch (1:12)



▶ Did the write do what you meant? (1:06)



▶ Embed or reference? Let the workload decide (1:21)

In PowerPoint Slide Show, click a thumbnail to play full screen. Embedded videos work offline.

MongoDB University and Colab

The Week 10 Lab 2 handout links to the class activities and the Day 2 notebook.

	MongoDB University	Our Colab notebook
Workspace	Tools provided by the activity	Python cells in a notebook you save
Guidance	Follow the prompts when a workspace is available	Edit the course cells and explain the results
Database	Use the environment specified by the lesson	Day 2 uses local practice data

Day 2: Open the Week 10 Lab 2 handout

Optional orientation: Introduction to the IDE

Optional practice: Atlas Data Explorer

Return to slide 4

Relationship cardinality and stored shape

Relationship in this application	Cardinality	A modeling consequence
Ticket to requester	Many tickets to one user	A shared user can change independently
Ticket to events	One ticket to many events	History can outgrow an embedded array
Publisher to headquarters	One to one, by assumption	A small owned address can fit in its publisher

Cardinality describes how entities relate. It does not choose the storage by itself. Ownership, reads, writes, and growth still matter.

One publisher and its headquarters

A one-to-one example
One publisher owns one small address.

```
{
  "publisher_id": "p7",
  "name": "Example Press",
  "headquarters": {"city": "New York", "country": "US"}
}
```

Embedding returns this address with its publisher in one read.
An independently maintained address may warrant its own document.